

Session 4 · Deck 4.B2

Expectations: Quadrature, and Discretizing the Shock

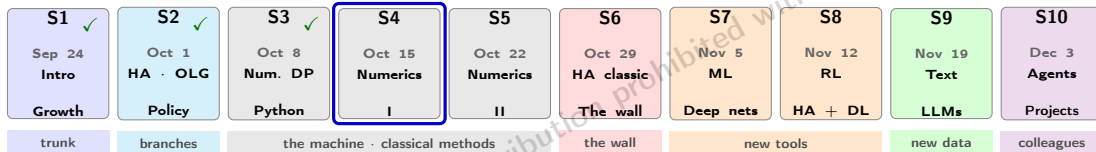
Zhigang Feng

Thursday, October 15, 2026

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are

you are here: 4.B2 ▼



Build it on paper → solve it on the machine → push it to the frontier with AI

Block B: 4.B1 approximation and interpolation · 4.B2 expectations: quadrature and the Markov chain · 4.B3 fast and accurate · 4.B4 the worked example, end to end.

An Expectation Is an Integral, and a Computer Can Only Add

Every Bellman sweep evaluates

$$\mathbb{E}[\hat{v}(k', z') \mid z] = \int \hat{v}(k', z') dF(z' \mid z)$$

at every state — an integral, inside the loop. A computer cannot integrate; it can only evaluate f at a point and add. Two questions sit under every solve we have run:

1. How does an integral become a **sum of a few function values**?
2. When the shock is **persistent**, where does the transition matrix Π come from?

$\mathbb{E}[e^z], z \sim N(0, 1)$	points	error
Gauss–Hermite	9	7×10^{-11}
trapezoid	17	9×10^{-9}
Monte Carlo	1 million	2×10^{-3}

Nine well-chosen points beat a million random ones.

The second question has teeth

Build a five-state chain for $\ln z' = 0.99 \ln z + 0.007 \varepsilon'$ the usual way and ask it what persistence it has. It answers $\rho = 1.0000$ — a **unit root** — with a standard deviation 36% too large. The other standard construction answers 0.990000, with an error of $6 \times 10^{-13}\%$.

Session 3 owed you this

3.A2 said the two constructions “disagree badly when $\rho > 0.95$, which is exactly the range macro uses”. This deck builds both and measures it.

The Idea: Replace the Integral by a Weighted Sum of a Few Points

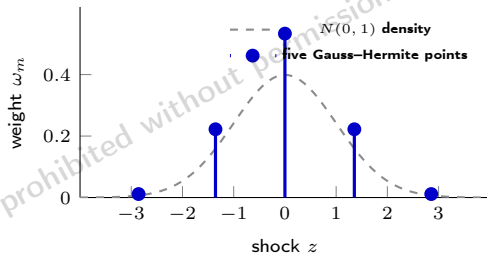
Every rule has the same shape: evaluate at a few **nodes**, add up with **weights**,

$$\mathbb{E}f(z) = \int f(z) w(z) dz \approx \sum_{m=1}^n \omega_m f(z_m).$$

Two choices — **where the nodes go, what the weights are** — and three ways to make them:

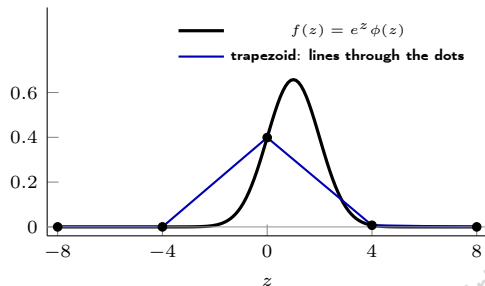
- **equally spaced** nodes, weights from a polynomial fit: Newton–Cotes;
- nodes and weights **chosen together** for the density: Gaussian rules;
- **random** nodes, equal weights: Monte Carlo.

For an expectation, w is the **density**. A rule that knows it puts the points where the probability is, not over a region that is mostly empty.



Five points for $z \sim N(0, 1)$: a small **discrete distribution** standing in for the continuous one. The weights 0.011, 0.222, 0.533, 0.222, 0.011 are probabilities and sum to one; an expectation is a five-term sum.

Newton–Cotes: What “Fit a Polynomial” Means



Five equally spaced nodes on $[-8, 8]$, $h = 4$. The area under the blue lines is the trapezoid estimate.

We cannot integrate f exactly, but we can integrate a **polynomial** exactly. So:

1. evaluate f at the nodes (the dots);
2. between nodes, replace f by the polynomial that **passes through those values** — the simplest choice is a straight line through each pair of neighbours, the **trapezoid rule**;
3. integrate that polynomial, which is just adding up areas.

Step 3 is a weighted sum of the $f(z_m)$: the weights are the areas of the pieces, $h(\frac{1}{2}, 1, \dots, 1, \frac{1}{2})$.

Five nodes: trapezoid 1.625 against the exact 1.649. Put the nodes closer together and the lines hug the curve: on a bell-shaped integrand like this one, **17 points already give** 9×10^{-9} .

Gauss–Hermite: Let the Density Choose the Points

Equally spaced nodes ignore where the probability is.

Instead choose **nodes and weights together**: $2n$ numbers, enough to make the rule **exact for every polynomial up to degree $2n - 1$** — twice what equally spaced nodes deliver. For a normal shock the nodes that do it are tabulated (**Gauss–Hermite**): they sit where the density lives, and the weights are **probabilities**.

n	error in $\mathbb{E}[e^z]$
3	1.1×10^{-2}
5	4.2×10^{-5}
9	7.2×10^{-11}
17	0

The recipe, for $z \sim N(\mu, \sigma^2)$

Standard-normal nodes $\hat{z}_m$ and probabilities ω_m from the table, then

$$\mathbb{E}f(z) \approx \sum_{m=1}^n \omega_m f(\mu + \sigma \hat{z}_m).$$

A smooth f is close to a polynomial, so $n = 5$ to 9 is usually enough.

A Gauss rule built for a *flat* weight (Gauss–Legendre on $[-8, 8]$) spends its nodes on empty tails: 17 nodes, off by 3×10^{-4} . Chebyshev's lesson again (4.B1): **put the points where they do the most good**.

Three Nodes by Hand: The Weights Are Probabilities

Why probabilities? The rule is exact for every polynomial up to degree $2n - 1$, so in particular for the constant $f = 1$:

$$\sum_m \omega_m = \int 1 \cdot w(z) dz = 1$$

when w is a density. Positive weights summing to one **are** a probability distribution: the rule is a small discrete shock standing in for the normal.

node $\hat{z}_m$ ($z \sim N(0, 1)$)	probability ω_m
$-\sqrt{3} = -1.732$	1/6
0	2/3
$\sqrt{3} = 1.732$	1/6

Ledger `gauss_weights`. NumPy's `hermgauss(n)` gives x_m, w_m for e^{-x^2} ; $\hat{z}_m = \sqrt{2} x_m, \omega_m = w_m / \sqrt{\pi}$.

Check it: the moments of $N(0, 1)$

$\mathbb{E}z^2 = 2 \cdot \frac{1}{6} \cdot 3 = 1$ and $\mathbb{E}z^4 = 2 \cdot \frac{1}{6} \cdot 9 = 3$: **exact**, as promised for degree ≤ 5 .

$\mathbb{E}z^6 = 2 \cdot \frac{1}{6} \cdot 27 = 9$ against the true 15: degree $6 > 2n - 1$, not promised. Five nodes get 15, and 105 for z^8 , right too.

That is the whole mechanism: three points reproduce every moment of the normal up to the fifth, so an f close to a fifth-degree polynomial is integrated almost exactly. On $\mathbb{E}[e^z]$: 1.1×10^{-2} with three nodes, 4.2×10^{-5} with five, 7.2×10^{-11} with nine.

Monte Carlo: Slow, and Indifferent to Dimension

Draw $z_1, \dots, z_n$ from the distribution and average — every node gets the same weight $1/n$:

$$\mathbb{E}[f(z)] \approx \frac{1}{n} \sum_m f(z_m), \quad \text{s.e.} = \frac{\sigma_f}{\sqrt{n}}.$$

n	error	s.e.
100	4.0×10^{-2}	1.7×10^{-1}
10,000	8.9×10^{-3}	2.0×10^{-2}
1,000,000	2.4×10^{-3}	2.2×10^{-3}

A hundredfold increase in draws buys one decimal digit.

Why: the errors of independent draws partly cancel, so the average's spread shrinks like $1/\sqrt{n}$ — and a draw is one point however many coordinates it has, so the dimension never enters.

Ledger quadrature.

And yet it wins, eventually

$n^{-1/2}$ **does not depend on the dimension.** A Gauss rule in d dimensions needs n^d nodes; Monte Carlo needs the same n whether d is 1 or 100.

So the choice is a property of d , not of taste: with a few shocks use a Gaussian rule; with many you have no choice — the regime Sessions 7 and 8 live in. **Quasi-Monte Carlo** (Sobol, Halton) replaces random draws by an evenly spread sequence and does better in moderate dimensions.

When the Shock Is Persistent, the Points Need Transition Probabilities

Gauss–Hermite handles $\mathbb{E}[f(z')]$ when z' is drawn afresh each period. In every model we have solved, z is **persistent**:

$$\ln z' = \rho \ln z + \sigma \varepsilon',$$

so tomorrow's distribution **depends on today's state**. The object we need is a whole conditional law for each today-state: a grid $\{z_1, \dots, z_n\}$ and a matrix Π with

$$\Pi_{jm} = \Pr(z' = z_m \mid z = z_j).$$

This is *not* the quadrature problem of the last frames, where the distribution was fixed; it moves with the state. We have written Π down by hand twice in Session 3. Where does it come from?

What a good chain should match: the persistence ρ , the unconditional variance $\sigma^2/(1 - \rho^2)$, and ideally the shape of the conditional distribution.

Why ρ near one is hard

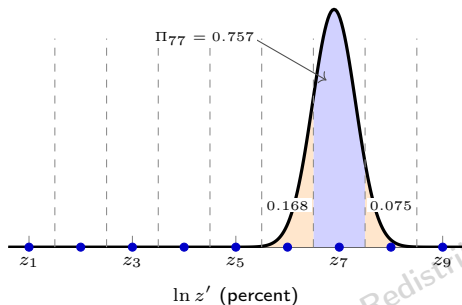
The unconditional standard deviation is

$$\sigma_z = \frac{\sigma}{\sqrt{1 - \rho^2}},$$

which **explodes** as $\rho \rightarrow 1$: at $\rho = 0.99$ it is seven times σ . A fixed number of states has to cover a much wider range, with each step correspondingly coarser.

Two standard constructions follow: **Tauchen**, which starts from the grid, and **Rouwenhorst**, which starts from the moments. The hook showed what the first delivers at $\rho = 0.99$.

Tauchen (1986): The Idea in One Picture



$n = 9$, $\rho = 0.95$, today at z_7 : tomorrow is $N(\rho z_7, \sigma^2)$.
Dashed lines: the cell edges, halfway between grid points.

Today the state is z_j . Tomorrow's $\ln z'$ is normal with mean ρz_j and standard deviation σ — the bell.

Cut the line **halfway between grid points**; each cell belongs to one grid point. Π_{jm} is the area of the bell over z_m 's cell:

$$\begin{aligned}\Pi_{jm} &= \Pr \left\{ z_m - \frac{\Delta}{2} < \rho z_j + \sigma \epsilon' \leq z_m + \frac{\Delta}{2} \right\} \\ &= F\left(\frac{z_m + \Delta/2 - \rho z_j}{\sigma}\right) - F\left(\frac{z_m - \Delta/2 - \rho z_j}{\sigma}\right).\end{aligned}$$

From z_7 : stay with probability 0.757, step down to z_6 with 0.168, up to z_8 with 0.075. The bell's mean ρz_7 sits a little below z_7 , so down is likelier than up — that is mean reversion.

Tauchen: The Algorithm, Step by Step

Inputs: ρ , σ , the number of states n , the coverage κ .

1. Stationary spread: $\sigma_z = \sigma / \sqrt{1 - \rho^2}$.
2. Endpoints: $\pm \kappa \sigma_z$; $\kappa = 3$ covers 99.7% of the stationary distribution.
3. Equally spaced grid: $z_m = -\kappa \sigma_z + (m - 1) \Delta$, $\Delta = \frac{2\kappa \sigma_z}{n - 1}$.
4. Cell edges, the midpoints: $\bar{z}_m = \frac{1}{2}(z_m + z_{m+1})$, $m = 1, \dots, n - 1$.
5. Cells: $I_1 = (-\infty, \bar{z}_1]$, $I_m = (\bar{z}_{m-1}, \bar{z}_m]$, $I_n = (\bar{z}_{n-1}, \infty)$.
6. Probabilities, with F the standard normal cdf:
$$\Pi_{jm} = F\left(\frac{\bar{z}_m - \rho z_j}{\sigma}\right) - F\left(\frac{\bar{z}_{m-1} - \rho z_j}{\sigma}\right),$$
$$\Pi_{j1} = F\left(\frac{\bar{z}_1 - \rho z_j}{\sigma}\right), \quad \Pi_{jn} = 1 - F\left(\frac{\bar{z}_{n-1} - \rho z_j}{\sigma}\right).$$

The two end cells are open, so they **absorb the tails**.

Conventions here

Zero mean. With a mean μ , the process is $\ln z' = \mu(1 - \rho) + \rho \ln z + \sigma \varepsilon'$: centre the grid on μ and subtract $\mu(1 - \rho)$ inside every F . We write the coverage κ and the cdf F because this course reserves λ and Φ .

Check before you use it

Every row sums to one; every entry is non-negative; and the chain's own ρ and σ_z , computed from its invariant distribution, sit next to the targets.

Why Tauchen Gets Stuck When ρ Is Near One

The chain leaves z_j only if the shock carries $\ln z'$ **past half a cell**. Measure half a cell in shocks:

$$\frac{\Delta/2}{\sigma} = \frac{\kappa \sigma_z}{(n-1)\sigma} = \frac{\kappa}{(n-1)\sqrt{1-\rho^2}}.$$

As $\rho \rightarrow 1$ the grid spreads with σ_z , but the shock σ does not grow. Each cell becomes many shocks wide, and only a tail event moves the chain.

At $\rho = 0.99$, $n = 5$, leaving the middle state takes a 5.3σ shock: probability 1.1×10^{-7} , about once in nine million quarters. That is the **unit root** of the first frame.

ρ	n	half cell, in σ	leave middle
0.90	5	1.72	0.085
0.95	5	2.40	0.016
0.95	9	1.20	0.230
0.99	5	5.32	1.1×10^{-7}
0.99	9	2.66	0.008
0.99	15	1.52	0.129

More states shrink the cell, but only like $1/(n-1)$. The grid was chosen **before** the persistence was taken into account. Rouwenhorst instead builds the probabilities so that the moments come out right by construction.

Ledger tauchen_detail, $\kappa = 3$, $\sigma = 0.007$.

Rouwenhorst (1995), Step 1: Two States Already Match the AR(1)

Two states, $\pm\psi$, and a symmetric chain:

$$\Pi_2 = \begin{pmatrix} p & 1-p \\ 1-p & p \end{pmatrix}.$$

Start at $z = \psi$ and compute tomorrow's moments:

$$\mathbb{E}[z' | z] = p\psi - (1-p)\psi = (2p-1)z,$$

$$\text{Var}[z' | z] = \psi^2 - (2p-1)^2\psi^2.$$

Choose the two numbers:

- $p = \frac{1+\rho}{2}$ makes the conditional mean ρz — the AR(1)'s;
- $\psi = \sigma_z$ makes the conditional variance $(1-\rho^2)\sigma_z^2 = \sigma^2$ — the AR(1)'s.

By symmetry the same holds at $z = -\psi$, and the chain spends half its time in each state.

Rouwenhorst (1995). Ledger rouwenhorst_detail.

moment	two-state chain
conditional mean ρz	exact
conditional variance σ^2	exact
unconditional mean 0	exact
unconditional variance σ_z^2	exact
autocorrelation ρ	exact $(2p-1)$
shape: a bell	no — two points

At $\rho = 0.95$: $p = 0.975$, $\psi = \sigma_z = 0.0224$, and a state lasts $1/(1-p) = 40$ quarters on average. Two points already get the first two moments right; more states only refine the **shape**.

Rouwenhorst, Step 2: From 2 to n States, One Coin at a Time

Grow the chain recursively. Given Π_{n-1} ,

$$\Pi_n = p \begin{pmatrix} \Pi_{n-1} & 0 \\ 0 & 0 \end{pmatrix} + (1-p) \begin{pmatrix} 0 & \Pi_{n-1} \\ 0 & 0 \end{pmatrix} \\ + (1-p) \begin{pmatrix} 0 & 0 \\ \Pi_{n-1} & 0 \end{pmatrix} + p \begin{pmatrix} 0 & 0 \\ 0 & \Pi_{n-1} \end{pmatrix},$$

then halve the interior rows: each is reached by two of the blocks, so it sums to two. The grid is equally spaced on $[-\psi, \psi]$, now with $\psi = \sigma_z \sqrt{n-1}$.

For $n = 3$:
$$\Pi_3 = \begin{pmatrix} p^2 & 2p(1-p) & (1-p)^2 \\ p(1-p) & p^2 + (1-p)^2 & p(1-p) \\ (1-p)^2 & 2p(1-p) & p^2 \end{pmatrix}.$$

What the recursion is doing

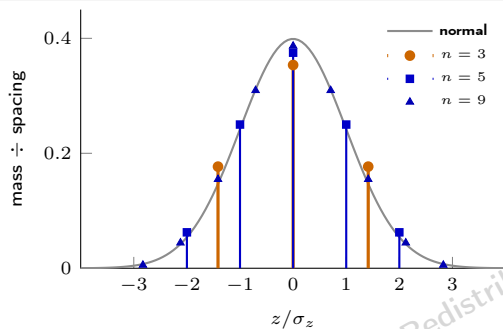
Take $n-1$ **independent copies** of Step 1's two-state chain, each worth $\pm\psi/(n-1)$, and add them up. The state is how many copies are high.

- Each copy has conditional mean ρ times itself, so the sum does too: **persistence** ρ .
- Independent variances add:
 $(n-1)\left(\frac{\psi}{n-1}\right)^2 = \frac{\psi^2}{n-1}$, which is σ_z^2 exactly when $\psi = \sigma_z \sqrt{n-1}$.

Each step of the recursion **adds one coin**: it stays (p) or flips ($1-p$) — the four blocks.

Kopecky & Suen (2010) give this reading. Ledger rouwenhorst_detail: counting $n-1$ coins reproduces the recursion's Π_n to 10^{-15} , $n = 3, \dots, 6$.

What Each New State Adds: Detail and Tails



The invariant distribution is $\text{Binomial}(n-1, \frac{1}{2})$ and tends to the normal.

Each added state does two things at once: the grid **reaches further** ($\pm\sqrt{n-1}\sigma_z$) — tails — and its spacing **shrinks** ($2\sigma_z/\sqrt{n-1}$) — detail.

n	end	spacing	$\text{Pr}(\text{end})$	kurtosis
2	1.00	2.00	0.5	1.00
3	1.41	1.41	0.25	2.00
5	2.00	1.00	0.0625	2.50
9	2.83	0.71	0.0039	2.75
15	3.74	0.53	6×10^{-5}	2.86

In units of σ_z . $\text{Pr}(\text{end}) = 2^{-(n-1)}$; kurtosis $3 - \frac{2}{n-1}$, the normal's is 3.

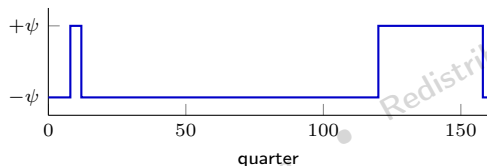
What never changes with n : ρ and σ_z are **exact**.
What does: the shape is binomial, with tails thinner than the normal's. Irrelevant in a linear world; in a strongly non-linear one it can matter.

Simulating a Chain: Start With Two States

Two states, $\pm\psi$. Whichever state you are in today, draw $u \sim U(0, 1)$:

stay if $u < p$, switch if $u \geq p$.

Repeat each period with a fresh u . That is the whole algorithm.



The first 160 quarters at $\rho = 0.95$ ($p = 0.975$): four switches.

10^6 quarters, $\rho = 0.95$

	simulated	AR(1)
sd of $\ln z$	0.022417	0.022418
autocorrelation	0.9498	0.95
switches per 100	2.51	$100(1 - p) = 2.5$

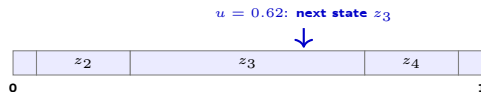
A long simulation recovers the moments Step 1 built in. Long spells, rare switches: that is what persistence 0.95 looks like with two states.

Fix the seed, and reuse the same draws across experiments.

`Ledger two_state_simulation; seed 11, burn-in 1,000.`

Simulating n States: The Same Draw, More Pieces

With two states, the high row cuts $[0, 1]$ at p : u below it means stay. With n states, row j cuts $[0, 1]$ into n pieces of length $\Pi_{j1}, \dots, \Pi_{jn}$; the piece u lands in is tomorrow's state.



- 1: $C \leftarrow$ cumulative sums along each row of Π
- 2: $j_0 \leftarrow$ the middle state
- 3: **for** $t = 0, \dots, T - 1$ **do**
- 4: draw $u_t \sim U(0, 1)$
- 5: $j_{t+1} \leftarrow$ first m with $C_{j_t m} \geq u_t$ $\triangleright$ searchsorted
- 6: **end for**
- 7: drop a burn-in; the path is z_{j_t}

Ledger markov_simulation; seed 11, burn-in 1,000. See also Sargent & Stachurski, *QuantEcon*, "Finite Markov Chains".

sd of $\ln z$, 10^6 quarters, same draws

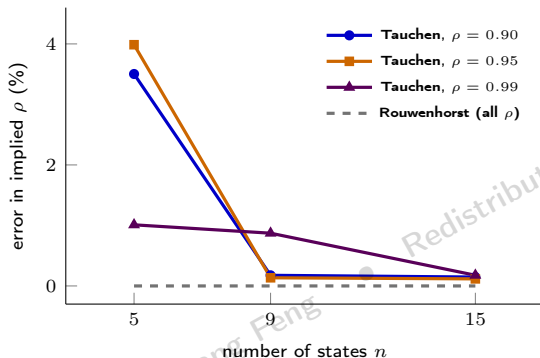
	simulated	chain	AR(1)
Rouwenhorst 7, 0.95	0.0224	0.0224	0.0224
Tauchen 7, 0.95	0.0277	0.0277	0.0224
Tauchen 5, 0.99	0	0.0676	0.0496

chain: from Π 's invariant distribution; $AR(1)$: σ_z .

It reproduces the chain, not the AR(1)

Tauchen's simulated sd is 23% too large at $\rho = 0.95$. At 0.99 the five-state chain **never left its middle state** in a million quarters — the 1.1×10^{-7} of the Tauchen frame.

The Comparison Session 3 Promised



Error in the chain's implied persistence, as a percentage of the target ρ .

ρ	n	Tauchen	Rouwen.
0.90	5	3.50%	2×10^{-14}
0.95	5	3.99%	0
0.99	5	1.01%	0
0.99	15	0.18%	7×10^{-14}

Rouwenhorst is exact to machine precision at **every** n and **every** ρ . The two lines are not close; one of them is zero.

And the Variance Is Worse Than the Persistence

Persistence is the forgiving statistic. The unconditional standard deviation — which is what sets the *size* of every business-cycle moment — is where Tauchen really suffers.

sd error		Tauchen	Rouwenhorst
$\rho = 0.90$	$n = 5$	26.9%	≈ 0
$\rho = 0.95$	$n = 5$	31.4%	≈ 0
$\rho = 0.99$	$n = 5$	36.2%	≈ 0
$\rho = 0.99$	$n = 9$	28.5%	≈ 0
$\rho = 0.99$	$n = 15$	20.1%	≈ 0

Target $\sigma_z = \sigma / \sqrt{1 - \rho^2}$; chain moments computed from the invariant distribution of each Π .

Read the last row

Even with **fifteen** states, at $\rho = 0.99$, the Tauchen chain's unconditional standard deviation is a **fifth too large**.

A model calibrated with that chain will overstate the volatility of output — and the natural response, lowering σ until the moments match, hides a discretization error inside a structural parameter.

What To Actually Do

Method	Use it when
<i>a fresh shock each period: a quadrature rule</i>	
Gauss–Hermite	normal shocks — the macro default ; 5 to 9 nodes
Monte Carlo / QMC	many shocks at once, or sampling only
<i>a persistent shock: a Markov chain</i>	
Rouwenhorst	the default , and essential for $\rho > 0.9$
Tauchen	low persistence, or when the conditional distribution must look normal
Tauchen–Hussey	quadrature-based; accurate for moderate ρ , degrades badly near one

Kopecky & Suen, “Finite State Markov-Chain Approximations to Highly Persistent Processes”, *RED* (2010), make the case formally: Rouwenhorst matches the first and second moments and the persistence exactly, for any n .

The habit to build

Whatever you use, **report what the chain actually delivers**: its implied ρ and σ_z next to the targets. It is three lines of code and it is the only way anyone can tell whether your moments came from the economics or the grid.

Homework 1 Part B asks for exactly that table, at $n = 5, 7, 9, 15$, — and then asks you to solve the model with both and see whether it moved the answer.

Takeaways

- Every quadrature rule is **nodes and weights**: an integral becomes a sum of a few function values. The only questions are where the nodes go and whether the rule knows the density.
- **Gauss–Hermite** chooses both: exact for polynomials up to degree $2n - 1$, nodes where the density lives, weights that are **probabilities**. Nine nodes beat a million Monte Carlo draws; Monte Carlo's $n^{-1/2}$ is the only option once the shocks are many.
- A persistent shock needs a **chain**, not a rule: a grid plus Π . It is an approximation with an error, and the error is measurable.
- **Tauchen** integrates the conditional normal over equally spaced cells. Near $\rho = 1$ a cell is many shocks wide, so the chain rarely moves: at $\rho = 0.99$ five states imply $\rho = 1.0000$ and a standard deviation 36% too large.
- **Rouwenhorst**: two states already match the mean, the variance and the persistence; each added state is one more coin. ρ and σ_z stay **exact** for every n .
- **Use Rouwenhorst**, and report the chain's implied ρ and σ_z beside the targets every time.

Bridge: Everything Is Now in Place

Session 3 ended with six crude choices and a list of what would repair each. We have now built all of them: a root-finder, a maximizer on an interval, derivatives, an interpolation scheme that keeps its shape, a quadrature rule, and a transition matrix that is not invented.

What we have not done is put them back together. The crude solver of 3.A4 took 1,352 sweeps and 2.4 billion comparisons at this session's calibration, and its worst Euler error was $10^{-1.5}$. The question for the next deck is simple: **how much of that was avoidable, and which repair bought the most?**

Next (4.B3)

Howard's policy improvement, monotonicity, concavity — and the endogenous grid method, done the way Carroll actually formulated it, which removes the root-finding entirely.

How each one works, and then one table: every technique, its speed-up and its accuracy, measured on the same model.